BTS CIEL — Mini-projet 2

Système de contrôle d'accès et de surveillance dynamique de la salle 215

Rapport technique — Tâche 3

Serveur, base de données, site web et intégration

Grégoire FEROUELLE

Carnus

Année 2026-2027

Sommaire

1. Présentation de la tâche 1

2. Analyse et conception 3

3. Réalisation 8

4. Tests et validation 11

5. Résultats et limites 19

6. Difficultés et solutions 21

7. Conclusion et perspectives 23

1. Présentation de la tâche

1.1. Rôle de la tâche dans le projet

Le mini-projet 2 a pour but de contrôler l'accès à la salle 215. Il est découpé en trois tâches : la tâche 1 gère le lecteur de badges RFID relié à un ESP32, la tâche 2 réalise le site web de réservation avec les QR codes, et la tâche 3, dont j'avais la charge, met en place le serveur qui relie l'ensemble.

Ma tâche est le point central du projet. Le lecteur de badges et le site web enregistrent et consultent tous leurs informations sur ce serveur. Sans lui, aucune des deux autres parties ne peut fonctionner avec les données réelles.

1.2. Objectifs

Les objectifs de ma tâche étaient les suivants :
- installer et administrer un serveur Linux dédié au projet ;
- concevoir et créer la base de données commune aux trois tâches ;
- mettre à disposition une API qui permet à l'ESP32 de vérifier un badge ;
- héberger le site web de la tâche 2 ;
- sécuriser le serveur, la base de données et les échanges ;
- assurer l'intégration et la cohérence de l'ensemble.

1.3. Périmètre

Ma tâche couvre tout ce qui se passe côté serveur : le système, la base de données, le serveur web, l'API, la sécurité et la supervision. Elle ne comprend pas le programme de l'ESP32 (tâche 1), ni les pages du site et la génération des QR codes (tâche 2). En revanche, c'est à moi de vérifier que ces éléments fonctionnent correctement une fois reliés au serveur.

1.4. Contraintes

Plusieurs contraintes ont orienté mes choix :
- le serveur devait fonctionner sous Linux et rester allumé en permanence dans la salle ;
- il devait être accessible sur le réseau de la salle, avec une adresse fixe validée par l'enseignant (plage au-dessus de .200) ;
- aucun mot de passe ni aucune clé ne devait apparaître dans les fichiers du projet, le dépôt GitHub étant public ;
- les communications sensibles devaient être chiffrées lorsque c'était possible ;
- j'ai réalisé cette tâche seul, ce qui m'a obligé à prioriser les fonctionnalités indispensables.

1.5. Interfaces avec les autres tâches

Tâche	Ce qu'elle échange avec le serveur	Comment
Tâche 1 (ESP32)	L'UID du badge lu, et la réponse autorisé ou refusé	API en JSON : POST /api/rfid.php, protégée par une clé
Tâche 2 (site web)	Les comptes, les réservations, les QR codes et le journal des actions	Accès direct à la base, le site étant installé sur le même serveur

Pour que tout le monde travaille sur la même structure, j'ai fourni aux deux autres tâches les scripts de création de la base et des données de test, disponibles sur le dépôt GitHub.

2. Analyse et conception

2.1. Besoin

Le serveur devait répondre à trois besoins principaux :

- stocker de manière fiable les utilisateurs, les badges, les réservations et l'historique des accès ;
- répondre rapidement à l'ESP32 lorsqu'un badge est présenté, pour que la porte réagisse sans attente ;
- héberger le site web de réservation et lui donner accès aux mêmes données.

Il devait aussi rester disponible en permanence dans la salle et être sécurisé, puisqu'il contient des mots de passe et décide de l'ouverture de la porte.

2.2. Choix du matériel

Critère	PC avec machine virtuelle	Raspberry Pi
Disponibilité	Dépend du PC : s'il est éteint ou emporté, le serveur est coupé	| Disponibilité | Dépend du PC : s'il est éteint ou emporté, le serveur est coupé | Reste dans la salle, allumé en permanence
Consommation	Élevée	Très faible
Puissance	Importante	Suffisante pour un seul site et une petite base
Coût	Utilise un PC entier	Faible

J'ai choisi le Raspberry Pi. Un serveur doit être disponible à tout moment, pour que les autres tâches puissent faire leurs tests et pour la démonstration finale. Sa puissance est largement suffisante pour le nombre d'utilisateurs prévu.

2.3. Choix des logiciels

Rôle	Choix	Autre solution envisagée	Raison du choix
Système	Raspberry Pi OS Lite (Debian 13)	Raspberry Pi OS avec bureau	Pas d'interface graphique inutile, plus de ressources pour les services
Serveur web	Apache	Nginx	Fonctionne directement avec PHP, configuration simple sous Debian
Langage de l'API	PHP	Python	Le site de la tâche 2 est en PHP : un seul langage sur le serveur
Base de données	MariaDB	PostgreSQL	Compatible avec XAMPP, utilisé par la tâche 2 pour développer

Le choix de PHP et MariaDB permet surtout à la tâche 2 de développer chez elle avec XAMPP, puis d'installer son site sur le serveur sans rien modifier.

2.4. Architecture

Le serveur est au centre du système. L'ESP32 de la tâche 1 et le site de la tâche 2 n'y accèdent pas de la même façon :
- l'ESP32 est un appareil séparé : il passe obligatoirement par l'API, qui vérifie une clé secrète avant de répondre. Il n'a jamais accès directement à la base ;
- le site web est installé sur le même serveur que la base : il y accède directement, en local, avec le même compte limité que l'API.

J'avais d'abord prévu de faire passer aussi le site par l'API. J'ai changé d'avis lorsque j'ai vu que le site de la tâche 2 appliquait déjà les règles de sécurité nécessaires (mots de passe hachés, requêtes préparées, contrôle des droits côté serveur). Refaire une API pour chaque action du site n'aurait rien apporté en sécurité, et aurait obligé à réécrire tout leur travail.

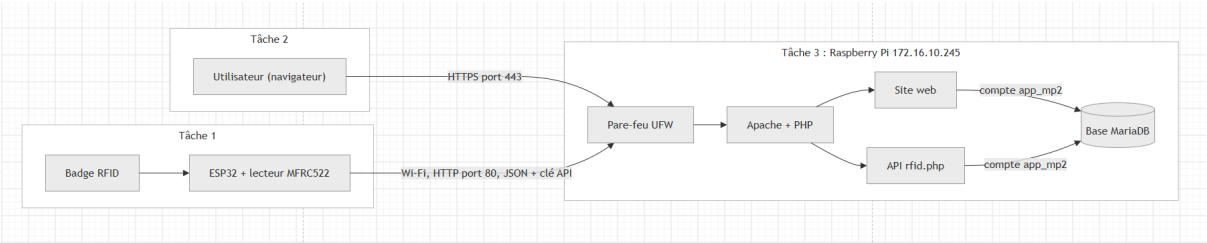


Figure 1 : Architecture du serveur

2.5. Conception de la base de données

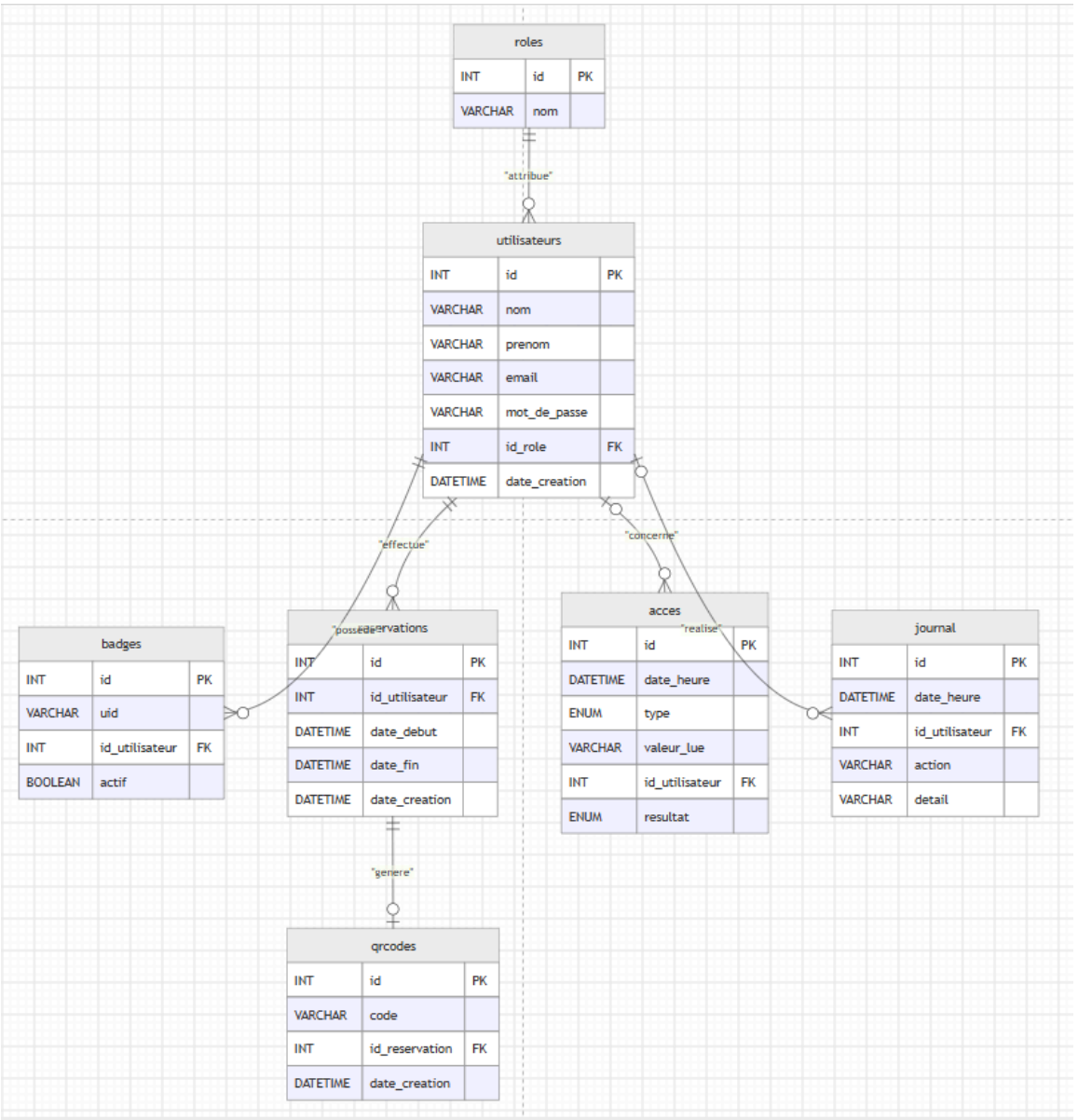


Figure 2 : Schéma de la base de données

La base contient sept tables. Les choix les plus importants sont les suivants :

- un badge est relié à un utilisateur, et une réservation est reliée à un utilisateur, pas au badge. C'est la personne qui réserve sur le site, et si elle perd son badge, ses réservations sont conservées ;
- il n'y a que deux rôles, admin et client. Tout nouveau compte est client par défaut, ce qui empêche de se créer soi-même un compte administrateur ;
- l'état de la salle (libre, réservée, occupée) n'est pas stocké : il est calculé à partir des réservations, pour éviter une information qui ne correspondrait plus à la réalité ;
- chaque tentative d'accès est enregistrée dans la table acces, même lorsque le badge est inconnu. La table journal garde la trace des actions importantes du site (créations et suppressions de comptes, annulations de réservations) ;
- les colonnes qui n'acceptent qu'un nombre limité de valeurs, comme le résultat d'un accès, sont de type ENUM : la base refuse toute autre valeur.

2.6. Conception de l'API

L'API reçoit l'UID du badge au format JSON et répond avec un résultat parmi quatre possibilités : autorise, inconnu, refuse ou erreur. Ces quatre cas correspondent à ceux demandés dans le cahier des charges.

La règle d'accès est la suivante : un administrateur est toujours autorisé, et un client seulement pendant l'une de ses réservations. Le prénom de la personne n'est renvoyé que si l'accès est autorisé, pour ne pas donner d'informations à quelqu'un qui testerait des badges au hasard.

3. Réalisation

3.1. Installation du serveur

J'ai installé Raspberry Pi OS Lite (64 bits) sur une carte microSD avec l'outil officiel Raspberry Pi Imager. Le serveur est relié au réseau de la salle par câble, et non en Wi-Fi, pour une connexion plus stable.

Une fois le système installé, je l'ai mis à jour et réglé sur le fuseau horaire Europe/Paris. Ce réglage est important, car toutes les dates enregistrées (accès, réservations) en dépendent. La synchronisation automatique de l'heure par internet est active.

Je contrôle le serveur à distance depuis mon PC en SSH. J'ai mis en place une connexion par clé, plus sûre qu'un simple mot de passe, et j'ai vérifié que la connexion directe avec le compte root est refusée.

Pour l'adresse IP, l'enseignant a autorisé la plage au-dessus de .200. Avant de choisir, j'ai testé les adresses de .240 à .254 avec un petit script : les adresses .250, .252 et .254 étaient déjà utilisées. J'ai donc choisi 172.16.10.245, que j'ai configurée en adresse fixe avec NetworkManager. Après un redémarrage, le serveur garde bien cette adresse et a toujours accès à internet.

(Commandes détaillées : voir annexe 1)

3.2. Base de données

J'ai installé MariaDB, puis je l'ai sécurisée : suppression des comptes anonymes et de la base de test, et connexion du compte administrateur réservée à l'administrateur du serveur. J'ai vérifié que MariaDB n'écoute que sur l'adresse locale (127.0.0.1) : la base est donc impossible à joindre depuis le réseau.

La base est créée par un script SQL (creation.sql), et un second script (donnees_test.sql) ajoute des données pour les tests. Grâce à ces deux fichiers, toute la base peut être recréée en quelques secondes, et les autres tâches ont pu l'installer à l'identique sur leurs propres postes.

J'ai créé un compte spécial pour les applications, app_mp2. Il peut seulement lire, ajouter, modifier et supprimer des lignes, mais pas toucher à la structure des tables. Son mot de passe n'apparaît dans aucun fichier du dépôt.

En cours de projet, il a fallu faire évoluer la base : remplacer les rôles admin, prof et élève par admin et client, puis ajouter la table journal. Pour la table journal, je l'ai ajoutée seule sur le serveur, sans relancer tout le script, afin de ne pas perdre l'historique des accès déjà enregistré.

(Scripts complets : voir annexe 2)

3.3. Serveur web et API

J'ai installé Apache avec PHP 8.4. Le mot de passe de la base et la clé de l'API sont rangés dans un fichier de configuration à part, /var/www/config.php. Ce fichier est placé en dehors du dossier du site, pour qu'il soit impossible de l'ouvrir depuis un navigateur, et seuls l'administrateur et Apache peuvent le lire. Sur GitHub, j'ai seulement déposé un modèle de ce fichier, sans les vraies valeurs.

L'API est composée de deux fichiers :
- bdd.php regroupe les fonctions communes : la connexion à la base, l'envoi des réponses en JSON avec le bon code HTTP, et la vérification de la clé API ;
- rfid.php traite les demandes de l'ESP32.

Quand l'ESP32 envoie un UID, rfid.php suit ces étapes :
1. il vérifie que la demande est bien de type POST, puis que la clé API est correcte ;
2. il remet l'UID en forme (majuscules, sans espaces ni deux-points), puis vérifie qu'il ne contient que des caractères autorisés ;
3. il cherche le badge et son propriétaire dans la base ;
4. il décide du résultat : un administrateur est toujours autorisé, un client seulement pendant l'une de ses réservations ;
5. il enregistre la tentative dans la table acces, puis renvoie le résultat à l'ESP32.

Toutes les requêtes SQL sont des requêtes préparées : l'UID n'est jamais inséré directement dans le SQL, ce qui empêche les injections. En cas d'erreur, le détail est écrit dans le journal d'Apache, et l'ESP32 reçoit seulement le résultat « erreur », pour ne pas révéler le fonctionnement interne du serveur.

(Code complet : voir annexe 3)

3.4. Sécurité du réseau

J'ai mis en place le pare-feu UFW. Par défaut, il bloque tout ce qui entre sur le serveur, sauf :
- le port 22 (SSH), autorisé seulement depuis le réseau de la salle ;
- le port 80 (HTTP), utilisé par l'ESP32 ;
- le port 443 (HTTPS), utilisé par le site.

J'ai aussi activé le HTTPS dans Apache. Le serveur n'étant pas accessible depuis internet, il ne peut pas obtenir de certificat d'une autorité reconnue : j'ai donc créé un certificat auto-signé, valable un an. Le navigateur affiche un avertissement, mais la connexion est bien chiffrée.

L'ESP32 communique encore en HTTP, car le HTTPS lui imposerait d'intégrer le certificat dans son programme. Cette limite est compensée en partie par la clé API et par le fait que les échanges restent sur le réseau de la salle.

(Commandes détaillées : voir annexe 4)

3.5. Supervision

Le Raspberry Pi ayant peu de ressources, j'ai préféré un script simple à un outil de supervision complet. Ce script, etat-serveur, affiche en une seule commande :
- l'état d'Apache, de MariaDB et du pare-feu ;
- la charge du processeur, la température, la mémoire et l'espace disque ;
- les cinq derniers accès et les trois dernières erreurs de l'API.

Je peux aussi afficher les accès en temps réel : l'écran se met à jour toutes les deux secondes, et chaque passage de badge apparaît immédiatement.

3.6. Intégration avec les autres tâches

Avec la tâche 1, j'ai défini le format des échanges dès le début du projet : l'adresse de l'API, le format JSON envoyé et les quatre réponses possibles. Ils ont ainsi pu programmer l'ESP32 en parallèle, avant même que l'API existe. J'ai ensuite ajouté leurs deux badges réels dans la base, et nous avons testé ensemble le passage des badges.

Avec la tâche 2, j'ai fourni les scripts de la base pour qu'ils développent leur site sur leurs postes avec XAMPP, sur exactement la même structure. Je leur ai aussi donné la fonction à appeler pour enregistrer leurs actions importantes dans la table journal. Leur site sera ensuite installé sur le serveur, à la place de la page par défaut d'Apache.

4. Tests et validation

4.1. Méthode

J'ai testé chaque partie du serveur au fur et à mesure de sa mise en place, puis l'ensemble avec les autres tâches. J'ai utilisé quatre types de tests :
- des tests de l'API depuis le serveur avec la commande curl, qui permet d'envoyer exactement la même demande que l'ESP32 et de vérifier la réponse et le code HTTP ;
- des tests réels avec l'ESP32 et les badges de la tâche 1 ;
- des tests de sécurité depuis mon PC, pour vérifier ce qui est accessible et ce qui est bloqué ;
- une analyse du trafic réseau avec Wireshark.

Pour chaque test, j'ai noté le résultat attendu avant de le lancer, puis le résultat obtenu.

4.2. Tests de l'API

Ces tests ont été réalisés avec les données de test : Alice est administratrice, Emma est cliente, et le badge de Hugo est désactivé.

ID	Test	Résultat attendu	Résultat obtenu	Statut
T01	Badge d'un administrateur	autorise + prénom, code 200	autorise, Alice, 200	OK
T02	Même badge écrit avec des « : » en minuscules	autorise + prénom, code 200	autorise, Alice, 200	OK
T03	Badge inconnu	inconnu, code 200	inconnu, 200	OK
T04	Badge désactivé	refuse, code 200	refuse, 200	OK
T05	Client sans réservation en cours	refuse, code 200	refuse, 200	OK
T06	Demande sans clé API	code 401	401	OK
T07	UID invalide	code 400	400	OK
T08	Demande de type GET au lieu de POST	code 405	405	OK
T09	Client pendant sa réservation	autorise + prénom, code 200	autorise, Emma, 200	OK

J'ai ensuite vérifié que chaque tentative était bien enregistrée dans la table acces. Le test T02 montre que la mise en forme de l'UID fonctionne : envoyé sous la forme a3:f2:c4:10, il a été enregistré sous la forme A3F2C410. Les tests T06 à T08 n'apparaissent pas dans l'historique, ce qui est normal : ces demandes sont rejetées avant d'arriver à la base.

4.3. Tests réels avec l'ESP32

ID	Test	Résultat attendu	Résultat obtenu	Statut
T15	Badge réel associé à un administrateur	Accès autorisé	Accès autorisé	OK
T16	Badge réel d'un client, pendant sa réservation	Accès autorisé	Accès autorisé	OK
T17	Même badge, juste après la fin de la réservation	Accès refusé	Accès refusé	OK
T18	Passage d'un badge après l'activation du pare-feu	Accès toujours possible	Accès possible	OK

Pour le test T17, j'ai réglé la fin de la réservation à une heure précise (12 h 13). Le badge était autorisé juste avant, puis refusé juste après : l'accès s'arrête exactement à la fin de la réservation.

4.4. Tests de la base de données

ID	Test	Résultat attendu	Résultat obtenu	Statut
T19	Suppression d'une table avec le compte de l'application	Refusée	Erreur 1142, refusée	OK
T20	Compte créé sans préciser de rôle	Rôle client	Rôle client	OK
T21	Réservation qui finit avant de commencer	Refusée	Erreur 4025, refusée	OK
T22	Ajout d'une ligne dans la table journal	Ligne enregistrée	Ligne enregistrée	OK

4.5. Tests de sécurité

ID	Test	Résultat attendu	Résultat obtenu	Statut
T10	Connexion SSH depuis le PC	Connexion possible	Connexion possible	OK
T11	Site en HTTP depuis le navigateur	Page affichée	Page affichée	OK
T12	Port 3306 (base de données) depuis le PC	Bloqué	Bloqué	OK
T13	Trace du blocage dans les journaux	Ligne UFW BLOCK	Ligne UFW BLOCK	OK
T14	Site en HTTPS depuis le navigateur	Page affichée en HTTPS	Page affichée	OK
T23	Connexion SSH avec le compte root	Refusée	Permission denied	OK

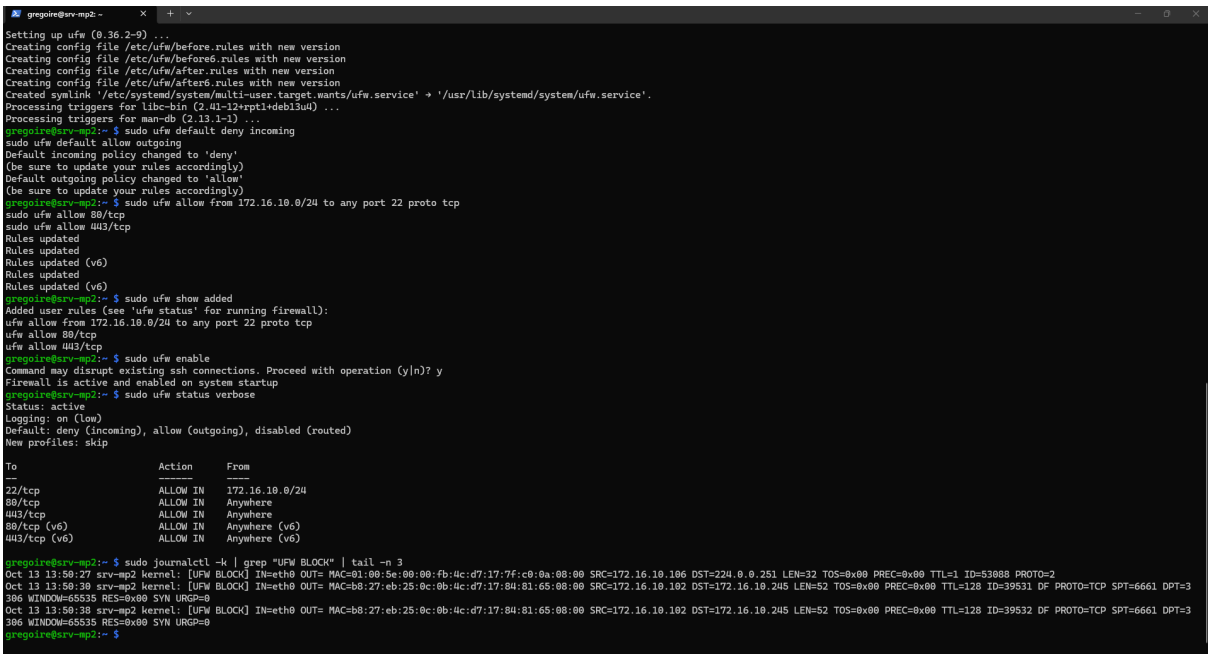


Figure 3 : Blocage par le pare-feu d'une tentative de connexion à la base de données (port 3306) depuis un PC du réseau

4.6. Analyse du trafic avec Wireshark

Sur un réseau d'école, un PC ne voit passer que son propre trafic. J'ai donc enregistré les échanges directement sur le serveur avec tcpdump, puis je les ai analysés sur mon PC avec Wireshark.

- En HTTP, entre l'ESP32 et l'API, tout est lisible : on voit l'UID du badge, la clé API et la réponse du serveur.
- En HTTPS, entre le navigateur et le site, on ne voit que des données chiffrées, illisibles.

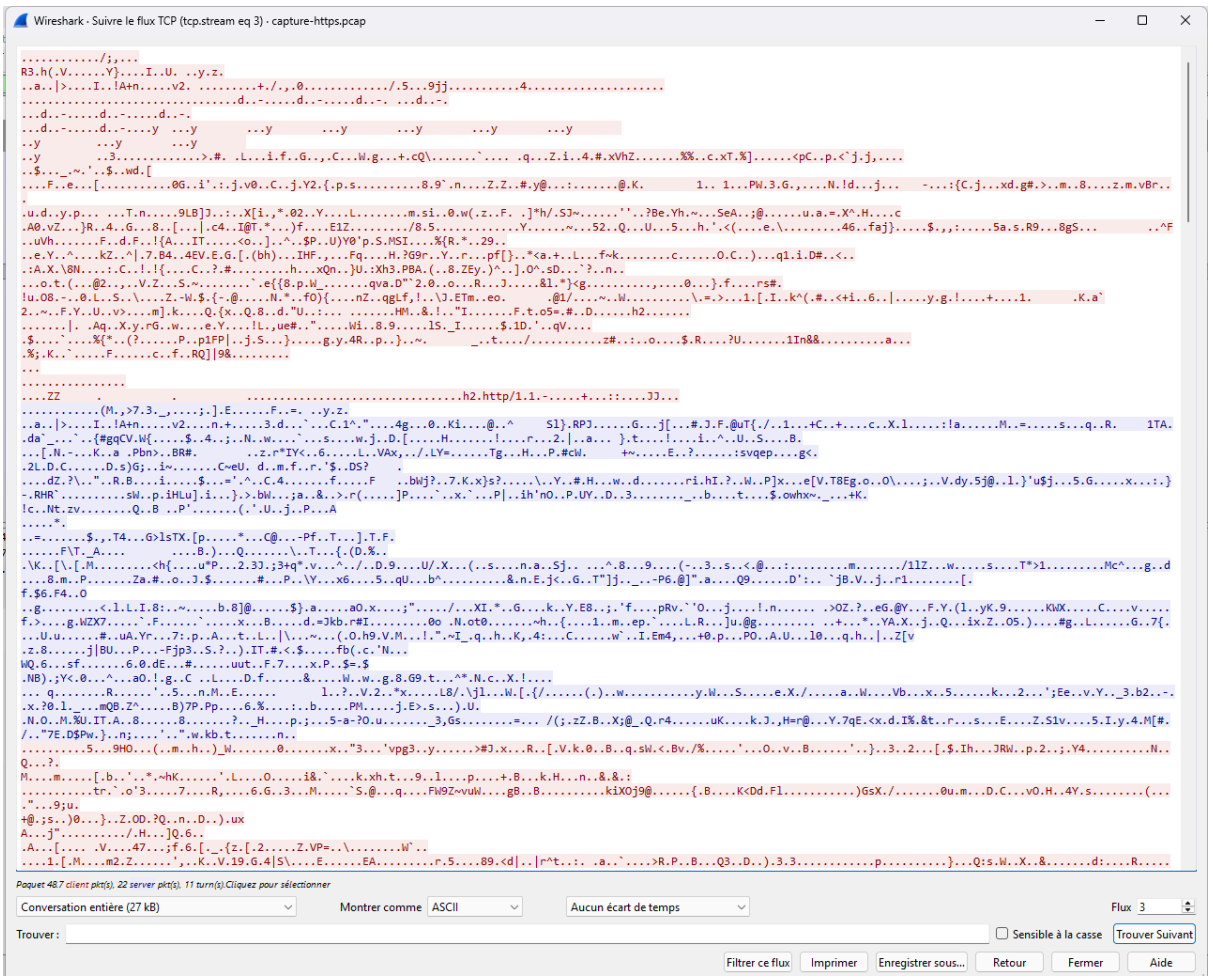


Figure 4 : Échange entre le navigateur et le site en HTTPS, analysé avec Wireshark : les données sont chiffrées

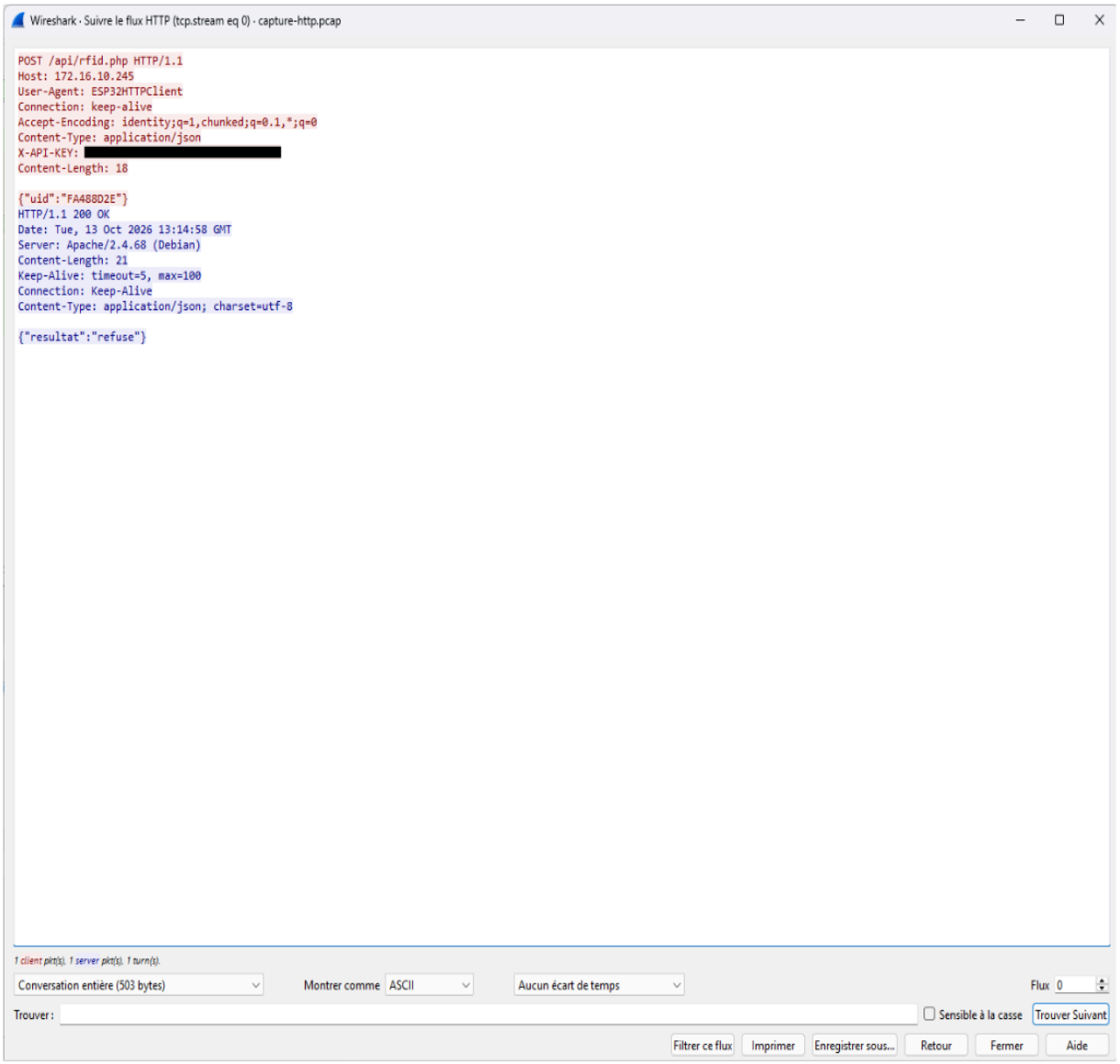


Figure 5 : Échange entre l'ESP32 et l'API en HTTP, analysé avec Wireshark (clé API masquée)

Cette comparaison confirme que le HTTPS fonctionne, et elle montre concrètement la principale limite de mon travail : la communication avec l'ESP32 pourrait être écoutée par quelqu'un connecté au même réseau.

4.7. Test final d'intégration

[À compléter après le test avec les trois tâches : création d'une réservation sur le site, puis passage du badge.]

4.8. Validation par rapport aux objectifs

Objectif	Validé par
Serveur Linux installé et administrable à distance	T10, T23
Base de données commune et protégée	T12, T19, T20, T21
API pour l'ESP32	T01 à T09, T15 à T17
Sécurité du serveur	T06, T12, T13, T14
Journalisation	Historique des accès, T13, T22
Intégration avec les autres tâches	T15 à T18, partie 4.7

5. Résultats et limites

5.1. Fonctionnalités réalisées

Fonctionnalité	État
Serveur Linux sur Raspberry Pi, administrable à distance en SSH par clé	Réalisé
Adresse IP fixe sur le réseau de la salle	Réalisé
Base de données commune aux trois tâches	Réalisé
Compte de l'application aux droits limités	Réalisé
API de vérification des badges pour l'ESP32	Réalisé
Règle d'accès : admin toujours, client pendant sa réservation	Réalisé
Historique de toutes les tentatives d'accès	Réalisé
Journal des actions importantes du site	Réalisé côté base, à brancher dans le site

Fonctionnalité	État
Pare-feu	Réalisé
HTTPS pour le site	Réalisé
Script de supervision du serveur	Réalisé
Installation du site de la tâche 2 sur le serveur	À finaliser

5.2. Mesures

Le script de supervision montre que le Raspberry Pi est largement suffisant pour ce projet :

Mesure	Valeur relevée
Charge du processeur	0,00 (serveur au repos)
Température	40 °C
Mémoire utilisée	271 Mo sur 905 Mo
Espace disque utilisé	3,2 Go sur 29 Go (12 %)
Temps de réponse de l'API	7 ms en moyenne (5 mesures, de 6,3 à 8,5 ms)

Le serveur garde donc beaucoup de marge pour accueillir le site de la tâche 2, ou d'autres lecteurs de badges plus tard.

5.3. Limites

Mon travail présente plusieurs limites, que j'ai identifiées et dont je connais les causes :

Limite	Conséquence	Raison
L'ESP32 communique en HTTP	Quelqu'un sur le réseau peut lire la clé API (vu avec Wireshark)	Le HTTPS sur ESP32 demande d'intégrer le certificat dans son programme
Certificat auto-signé	Le navigateur et le téléphone affichent un avertissement	Le serveur n'est pas accessible depuis internet
Une seule clé API	Si elle est découverte, il faut la changer dans le serveur et dans l'ESP32	Un seul lecteur de badges pour l'instant
Pas de sauvegarde automatique de la base	Si la carte SD tombe en panne, les données sont perdues	Non prévu dans le temps du projet
Un seul serveur	S'il tombe en panne, plus aucun badge ne fonctionne	Projet sur une seule salle
Le site n'est joignable que depuis le réseau de la salle	Le lien du QR code ne s'ouvre pas en 4G	Adresse IP privée

Ces limites n'empêchent pas le système de fonctionner dans le cadre du projet. Les améliorations possibles sont présentées dans la conclusion.

6. Difficultés et solutions

6.1. Choix de l'adresse IP fixe

Problème : au départ, le serveur recevait automatiquement l'adresse 172.16.10.225. Une adresse attribuée automatiquement peut changer, ce qui aurait obligé la tâche 1 à modifier le programme de l'ESP32 à chaque fois.

Cause : le réseau de l'école distribue les adresses automatiquement (DHCP). L'enseignant a autorisé la plage au-dessus de .200, mais l'adresse .225 reçue automatiquement montrait que cette plage était aussi distribuée par le réseau. Choisir une adresse au hasard risquait donc un conflit avec une autre machine.

Solutions envisagées :
- garder l'adresse automatique, mais sans garantie qu'elle reste la même ;
- choisir une adresse fixe dans la plage autorisée, après avoir vérifié qu'elle était libre.

Solution retenue : j'ai testé une première adresse, .250, qui a répondu : elle était déjà utilisée par une autre machine. J'ai alors testé toutes les adresses de .240 à .254 avec un petit script, et choisi .245, qui était libre. Cette vérification a une limite : une machine éteinte au moment du test n'apparaît pas. J'ai donc prévenu l'enseignant de l'adresse choisie.

6.2. Erreur 500 au premier test de l'API

Problème : au premier test, l'API répondait systématiquement « erreur » avec le code 500.

Cause : comme prévu dans mon code, le détail de l'erreur n'apparaissait pas dans la réponse. Je l'ai trouvé dans le journal d'Apache : « Access denied for user 'app_mp2' ». Le mot de passe écrit dans le fichier de configuration ne correspondait pas à celui du compte de la base.

Solution retenue : j'ai d'abord testé le mot de passe à la main, pour savoir si l'erreur venait du compte ou du fichier. Le compte fonctionnait : j'ai donc corrigé le mot de passe dans le fichier de configuration. Cette difficulté m'a montré l'intérêt de séparer les messages : un message simple pour l'ESP32, et un message détaillé dans le journal du serveur pour l'administrateur.

6.3. Désaccords sur la structure de la base

Problème : en cours de projet, la tâche 1 m'a proposé une autre structure de tables, dans laquelle le rôle et le prénom étaient enregistrés dans le badge, et les réservations reliées au badge.

Cause : la tâche 1 voyait la base du point de vue de l'ESP32, alors que la base devait aussi servir au site de la tâche 2, qui a besoin de comptes avec un email et un mot de passe.

Solutions envisagées :
- adopter leur structure, ce qui obligeait à réécrire l'API, à refaire les tests et à priver le site de comptes utilisateurs ;
- garder la structure existante, en vérifiant qu'elle répondait bien à leurs besoins.

Solution retenue : j'ai comparé point par point leur demande avec la base et l'API existantes. L'ESP32 n'accède jamais directement aux tables : il envoie un UID et reçoit une réponse, et

ces échanges correspondaient déjà exactement à ce qu'ils attendaient. Je leur ai envoyé la structure complète, et nous avons décidé ensemble de la garder.

6.4. Autres difficultés

Problème	**Cause**	**Solution retenue**
Impossible de se connecter au Raspberry après la première installation	Les réglages (utilisateur, SSH) n'avaient pas été appliqués par Raspberry Pi Imager	Création de l'utilisateur avec un écran et un clavier, puis activation de SSH avec raspi-config
Rôles de départ (admin, prof, élève) ne correspondant pas au besoin	Besoin précisé par les autres tâches en cours de projet	Passage à deux rôles, admin et client, dans les scripts et sur le serveur
Le script de supervision indiquait le pare-feu comme arrêté	Il vérifiait le service de démarrage d'UFW, et non le pare-feu lui-même	Vérification directe avec la commande ufw status
Le serveur téléchargeait l'ancienne version d'un fichier modifié sur GitHub	GitHub garde une copie des fichiers pendant quelques minutes	Ajout d'un paramètre à l'adresse pour forcer la nouvelle version, puis vérification du contenu
Première mesure du temps de réponse anormalement rapide	La clé API était vide : le serveur refusait la demande avant d'interroger la base	Vérification du code de réponse, puis nouvelle mesure avec la bonne clé

6.5. Organisation

J'ai réalisé cette tâche seul. Pour tenir les délais, j'ai donné la priorité à ce dont les autres tâches avaient besoin pour avancer : la base de données et le format des échanges avec l'API ont été fixés dès le début. J'ai aussi documenté chaque étape sur GitHub au fur et à mesure, avec ses résultats de tests, ce qui m'a permis de rédiger ce rapport à partir de traces fiables.

7. Conclusion et perspectives

7.1. Bilan

L'objectif de ma tâche était de mettre en place le serveur qui relie le lecteur de badges et le site de réservation. Cet objectif est atteint : le serveur fonctionne en permanence dans la salle, la base de données est commune aux trois tâches, et l'API répond à l'ESP32 en quelques millisecondes. Les tests réels avec les badges de la tâche 1 ont confirmé que la règle d'accès fonctionne : un administrateur entre toujours, et un client seulement pendant sa réservation, jusqu'à la minute près.

La sécurité a été prise en compte à chaque étape : connexion SSH par clé, pare-feu, base inaccessible depuis le réseau, compte de l'application aux droits limités, aucun mot de passe sur GitHub, et HTTPS pour le site. Chaque passage de badge est enregistré, ce qui permet de retracer tous les accès à la salle.

La dernière étape, l'installation du site de la tâche 2 sur le serveur, est en cours de finalisation. La base, le compte de l'application et le fichier de configuration sont déjà prêts pour l'accueillir.

7.2. Ce que ce projet m'a apporté

Ce projet m'a permis de mettre en place un serveur complet de A à Z, de l'installation du système jusqu'à la sécurité et à la supervision. J'ai surtout compris le rôle d'une tâche d'intégration : définir tôt la base et le format des échanges permet aux autres de travailler en même temps, sans attendre que tout soit terminé. J'ai aussi appris à vérifier systématiquement mes résultats, y compris quand ils paraissent bons, et à documenter au fur et à mesure plutôt qu'à la fin.

7.3. Améliorations possibles

Amélioration	**Intérêt**
Passer l'ESP32 en HTTPS	Empêcher la lecture de la clé API sur le réseau, principale limite actuelle
Une clé API par lecteur de badges, avec une table des lecteurs	Ajouter d'autres portes ou d'autres salles, et savoir quel lecteur a été utilisé
Enregistrer l'adresse de l'appareil dans l'historique des accès	Retrouver l'origine d'une demande suspecte
Sauvegarde automatique de la base chaque nuit	Ne pas perdre les données en cas de panne de la carte SD
Adresse de vérification du QR code dans l'API	Permettre à un lecteur à l'entrée de valider les QR codes

Amélioration	Intérêt
Alertes automatiques (service arrêté, disque plein)	Être prévenu d'un problème sans avoir à lancer le script de supervision
Limiter le nombre de demandes à l'API	Empêcher quelqu'un d'essayer des milliers de badges au hasard

L'architecture actuelle permet ces évolutions sans tout refaire : ajouter un lecteur ou une nouvelle fonction revient à ajouter une table ou une adresse à l'API, sans toucher au reste du système.